



TP2 - Java

🕒 Date de création	26 août 2026 15:21
☰ Étiquettes	

1.1 / 1.2

```
package tp2_java;

public class SinglyListClass {
    private Node header;
    public Integer size;

    public SinglyListClass() {
        header = null;
        size = 0;
    }

    public SinglyListClass(Node header) {
        this.header = header;
        size = 1;
    }

    public Node getHeader() {
```

```

        return header;
    }

    public void setHeader(Node header) {
        this.header = header;
    }

    public Integer size() {
        return size;
    }

    public boolean isEmpty() {
        return (header == null);
    }

    public Integer first() {
        return header.getElement();
    }

    public Integer last() {
        Node tmpHeader = header;
        while(tmpHeader.getNext() != null) {
            tmpHeader = tmpHeader.getNext();
        }
        return tmpHeader.getElement();
    }

    public void addFirst(Integer el) {
        Node newNode = new Node(el);
        newNode.setNext(header);
        header = newNode;
        size++;
    }

    public void addLast(Integer el) {
        Node newNode = new Node(el);
        Node tmpHeader = header;
        if (header == null) {

```

```

        header = newNode;
        return;
    }
    while(tmpHeader.getNext() != null) {
        tmpHeader = tmpHeader.getNext();
    }
    tmpHeader.setNext(newNode);
    size++;
}

public Integer removeFirst() {
    if(header == null) {
        return null;
    }
    Node tmpHeader = header;
    header = header.getNext();
    return tmpHeader.getElement();
}

public Integer secondeLast() {
    if(header == null || size() < 2) {
        return null;
    }
    if(size() == 2) {
        return header.getElement();
    }
    Node tmpHeader = header;
    Node nextTmpHeader = header.getNext();
    while(nextTmpHeader.getNext() != null) {
        tmpHeader = tmpHeader.getNext();
        nextTmpHeader = nextTmpHeader.getNext();
    }
    return tmpHeader.getElement();
}

public void reverse() {
    Node pred = null;
    Node curr = header;
    Node next = null;

```

```

        while (curr != null) {
            next = curr.next;
            curr.next = pred;
            pred = curr;
            curr = next;
        }
        header = pred;
    }

    public void switchNode(Node a, Node b) {
        if (a == b || a == null || b == null || head == null)
        {
            return "Incorrect";
        }

        Node prevA = null;
        currA = head;
        while (currA != null && currA != a) {
            prevA = currA;
            currA = currA.getNext();
        }

        Node prevB = null;
        currB = head;
        while (currB != null && currB != b) {
            prevB = currB;
            currB = currB.getNext();
        }

        if (currA == null || currB == null)
        {
            return "un des noeuds n'existe pas";
        }

        if (prevA != null)
        {
            prevA.setNext(b);
        }
        else head = b;
    }

```

```

        if (prevB != null)
        {
            prevB.setNext(a);
        }
        else head = a;

        Node temp = a.getNext();
        a.setNext(b.getNext());
        b.setNext(temp);
    }

    private static class Node {
        private Integer el;
        Node next;

        public Node getNext() {
            return next;
        }
        public void setNext(Node next) {
            this.next = next;
        }
        public Node(Integer el) {
            this.el = el;
            next = null;
        }
        public Integer getElement() {
            return el;
        }
        public String toString() {
            return el.toString();
        }
    }

    public static void main(String[] args) {
        SinglyListClass maListe=new SinglyListClass(nul
l);

        maListe.addLast(4);
    }

```

```

        System.out.println(maListe.getHeader());
    }
}

```

1.3 / 1.4

```

public class DoublyLinkedList {

    private Node head = null;
    private Node tail = null;
    private int size = 0;

    private static class Node {
        private Integer element;
        private Node pred;
        private Node next;

        public Node(Integer element, Node pred, Node next)
        {
            this.element = element;
            this.pred = pred;
            this.next = next;
        }

        public Integer getElement() {
            return element;
        }

        public Node getPred() {
            return pred;
        }

        public Node getNext() {
            return next;
        }

        public void setPred(Node pred) {
            this.pred = pred;
        }
    }
}

```

```

        public void setNext(Node next) {
            this.next = next;
        }
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public void addFirst(Integer el) {
        Node newNode = new Node(el, null, head);
        if (isEmpty()) {
            tail = newNode;
        }
        else {
            head.setPred(newNode);
        }
        head = newNode;
        size++;
    }

    public void addLast(Integer el) {
        Node newNode = new Node(el, tail, null);
        if (isEmpty()) {
            head = newNode;
        }
        else {
            tail.setNext(newNode);
        }
        tail = newNode;
        size++;
    }

    public Integer removeFirst() {
        if (isEmpty()) {
            return null;
        }
    }

```

```

    }
    Node temp = head;
    head = head.getNext();
    size--;
    if (isEmpty()) {
        tail = null;
    }
    else {
        head.setPred(null);
    }
    return temp.getElement();
}

public int nbNodes(Node head) {
    if (head == null) {
        return 0;
    }
    int nb = 1;
    Node curr = head.getNext();
    while (curr != null && curr != head) {
        nb++;
        curr = curr.getNext();
    }
    return nb;
}

public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    DoublyLinkedList other = (DoublyLinkedList) obj;
    if (this.size != other.size) {
        return false;
    }
    Node currentThis = this.head;
    Node currentOther = other.head;

    while (currentThis != null) {

```

```
        if (!java.util.Objects.equals(currentThis.getElement(), currentOther.getElement())) {  
            return false;  
        }  
        currentThis = currentThis.getNext();  
        currentOther = currentOther.getNext();  
    }  
  
    return true;  
}  
}
```

1.5